



Chapter 6: Introducing Classes

Object Oriented Programming Language (JAVA)

Md. Eamin Rahman



Class Fundamentals

- A **class** defines a **new data type** in Java.
- Once defined, a class can be used to create **objects**.
- A class acts as a **blueprint/template** for objects.
- An **object** is a specific **instance** of a class.
- The terms **object** and **instance** are often used interchangeably.

Declaring a Class

- A class is declared using the **class** keyword
- A class contains:
 - **Instance variables** → store data
 - **Methods** → define behavior
- **Variables + methods** together are called **members** of the class

General Form of a Class

```
class ClassName {  
    // Instance variables  
    type instance-variable1;  
    type instance-variable2;  
    // ...  
    type instance-variableN;  
  
    // Methods  
    type methodname1(parameter-list) {  
        // body of method  
    }  
    type methodname2(parameter-list) {  
        // body of method  
    }  
    // ...  
    type methodnameN(parameter-list) {  
        // body of method  
    }  
}
```

Instance Variables

- Variables defined inside a class are called **instance variables**
- Each object gets its **own copy** of these variables
- Object data remains **separate and independent**

Methods in a Class

- Methods define how class data is used
- Methods have a structure similar to `main()`
- Most methods are **not** declared as:
 - `static`
 - `public`

```
returnType methodName(parameterList) {  
    // method body  
}
```

Does Every Class Need `main()`?

- A Java class **does not need** a `main()` method
- `main()` is required only when:
 - the class is the **starting point** of the program
- Many Java applications use classes without `main()`

```
public static void main(String[] args) {  
    // program starts here  
}
```

A Simple Class Example

```
class Box {  
    double width;  
    double height;  
    double depth;  
}
```

- **Box** defines a **new data type**
- It contains three **instance variables**:
 - **width**
 - **height**
 - **depth**

A Simple Class Example

```
class Box {  
    double width;  
    double height;  
    double depth;  
}
```

- **Box** defines a **new data type**
- It contains three **instance variables**:
 - **width**
 - **height**
 - **depth**

⚠ Important

A class declaration only creates a *template*; it does not create an *actual object*.

Creating an Object

```
Box mybox = new Box();
```

- **new** creates an actual object in **memory**
- Thus, it will have “**physical**” reality.
- **mybox** is a **reference variable**
- **mybox** refers to an instance of **Box**

Part	Meaning
Box	Class type
mybox	Object reference
new Box()	Creates object

Accessing Instance Variables

- Use the **dot operator** `.` to access members of an object

```
mybox.width = 10;  
mybox.height = 20;  
mybox.depth = 15;
```

- Syntax:

```
object.member
```

Instance Variables Are Independent

```
class Car {  
    String color;  
}  
  
Car c1 = new Car();  
Car c2 = new Car();  
  
c1.color = "Red";  
c2.color = "Blue";
```

- **c1** and **c2** store different values independently

The Dot Operator

- Connects an object with its:
 - **variables**
 - **methods**

Examples

```
mybox.width  
mybox.getVolume()
```

- Commonly called the **dot operator**
- Officially classified in Java as a **separator**

A Program uses the **Box** Class

```
1 // A program that uses the Box class. Call this file BoxDemo.java
2 class Box {
3     double width;
4     double height;
5     double depth;
6 }
7 // This class declares an object of type Box.
8 class BoxDemo {
9     public static void main(String[] args) {
10         Box mybox = new Box();
11         double vol;
12         // assign values to mybox's instance variables
13         mybox.width = 10;
14         mybox.height = 20;
15         mybox.depth = 15;
16         vol = mybox.width * mybox.height * mybox.depth; // compute volume of box
17         IO.println("Volume is " + vol);
18     }
19 }
```

A Program uses the **Box** Class

```
1 // A program that uses the Box class. Call this file BoxDemo.java
2 class Box {
3     double width;
4     double height;
5     double depth;
6 }
7 // This class declares an object of type Box.
8 class BoxDemo {
9     public static void main(String[] args) {
10         Box mybox = new Box();
11         double vol;
12         // assign values to mybox's instance variables
13         mybox.width = 10;
14         mybox.height = 20;
15         mybox.depth = 15;
16         vol = mybox.width * mybox.height * mybox.depth; // compute volume of box
17         IO.println("Volume is " + vol);
18     }
19 }
```

Console Output

Volume is 3000.0

Multiple **Box** Objects

```
1  class BoxDemo2 {
2      public static void main(String[] args) {
3          Box mybox1 = new Box();
4          Box mybox2 = new Box();
5          double vol;
6          // assign values to mybox1's instance variables
7          mybox1.width = 10;
8          mybox1.height = 20;
9          mybox1.depth = 15;
10         // assign different values to mybox2's instance variables
11         mybox2.width = 3;
12         mybox2.height = 6;
13         mybox2.depth = 9;
14         // compute volume of first box
15         vol = mybox1.width * mybox1.height * mybox1.depth;
16         IO.println("Volume is " + vol);
17         // compute volume of second box
18         vol = mybox2.width * mybox2.height * mybox2.depth;
19         IO.println("Volume is " + vol);
20     }
21 }
```

Multiple **Box** Objects

```
1 class BoxDemo2 {
2     public static void main(String[] args) {
3         Box mybox1 = new Box();
4         Box mybox2 = new Box();
5         double vol;
6         // assign values to mybox1's instance variables
7         mybox1.width = 10;
8         mybox1.height = 20;
9         mybox1.depth = 15;
10        // assign different values to mybox2's instance variables
11        mybox2.width = 3;
12        mybox2.height = 6;
13        mybox2.depth = 9;
14        // compute volume of first box
15        vol = mybox1.width * mybox1.height * mybox1.depth;
16        IO.println("Volume is " + vol);
17        // compute volume of second box
18        vol = mybox2.width * mybox2.height * mybox2.depth;
19        IO.println("Volume is " + vol);
20    }
21 }
```

Console Output

```
Volume is 3000.0
Volume is 162.0
```

Declaring Objects: Two-Step Process

To create an object in Java:

1. **Declare** a reference variable
2. **Create** the object using **new**

```
Box mybox;           // Step 1: declaration  
mybox = new Box();   // Step 2: object creation
```

Step 1: Declare a Reference Variable

```
Box mybox;
```

- **mybox** is a reference variable
- It can refer to an object of type **Box**
- No object is created yet
- No memory for the object's data is allocated yet

Step 2: Create the Object

```
mybox = new Box();
```

- **new** creates an actual object
- Memory is allocated **at runtime**
- **new** returns a **reference** to the created object
- The reference is stored in **mybox**

Key Points About **new**

- **new** performs dynamic memory allocation
- Objects are created during program execution
- Class objects in Java are created using **new**
- The reference returned by **new** allows access to the object

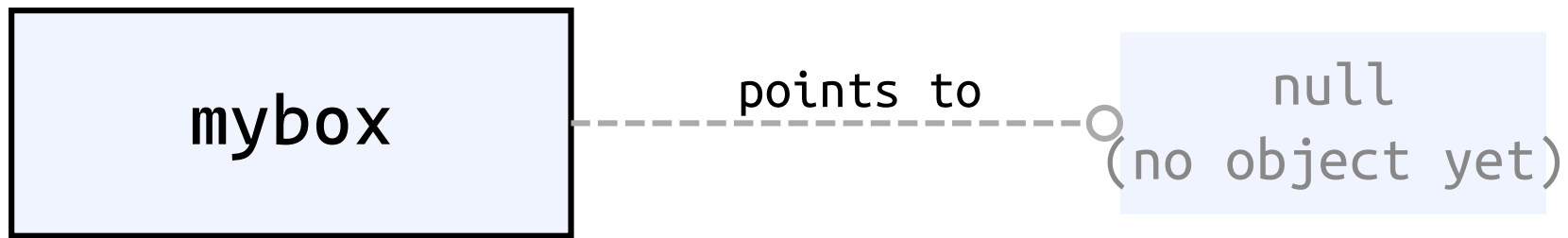
Common Shortcut

```
Box mybox = new Box();
```

- Combines declaration and object creation into one statement

Step 1 — Declaration

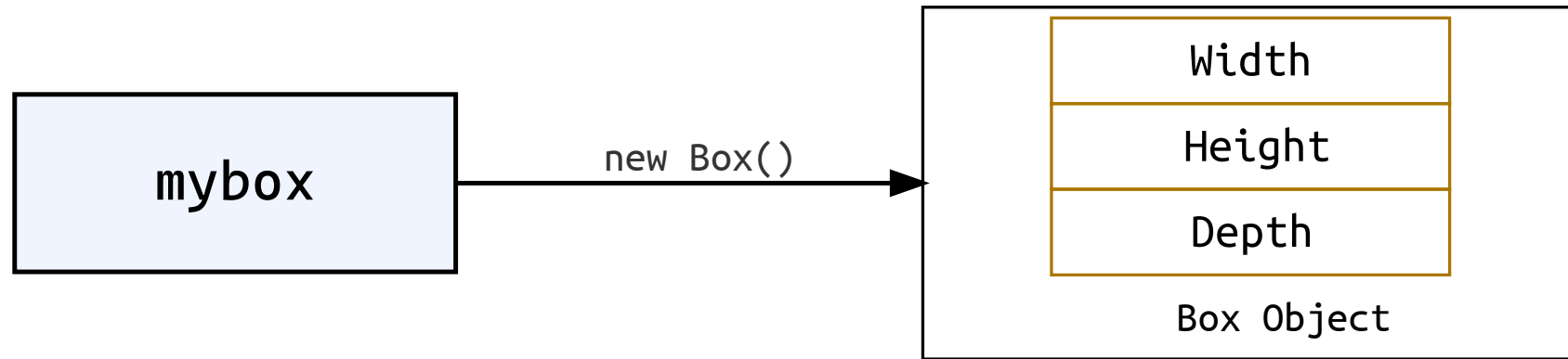
```
Box mybox;
```



- A reference variable `mybox` is created on the stack.
- It currently holds `null` — no object exists in memory yet.

Step 2 — Instantiation

```
mybox = new Box()
```



- `new Box()` allocates a **Box object** on the heap.
- `mybox` now holds a **reference (pointer)** to that object.

The **new** Operator

- **new** creates an object and allocates memory for it
- General syntax:

```
classVar = new ClassName();
```

Part	Meaning
classVar	Reference variable
ClassName	Class being instantiated
()	Invokes the constructor

What Happens When **new** Executes?

```
Box mybox = new Box();
```

1. **Memory** is allocated for the object
2. The class **constructor** is executed
3. A **reference** to the object is returned
4. The **reference** is stored in **mybox**

Constructors

- A **constructor** defines what happens when an object is created
- Constructor name is the *same* as the class name
- Constructors **initialize** object state

If No Constructor Is Defined

- Java automatically provides a **default constructor**

```
class Box {  
    double width;  
    double height;  
    double depth;  
}
```

- Since **no constructor** is written for **Box**, Java supplies a *default* one

Why Don't Primitive Types Use **new**?

Primitive Types

```
int count = 10;  
char grade = 'A';  
double price = 99.5;
```

- Primitive types are **not** objects
- Implemented as *regular* variables
- No object creation **overhead**

Dynamic Memory Allocation

- Objects are created **at runtime**
- Programs can create as **many objects** as needed
- **Memory** is allocated only when an object is *created*

Advantage

- Flexible and efficient memory usage

Limitation

- Since available memory is finite, if memory cannot be allocated, a **runtime exception** occurs

Objects vs Primitive Types

Primitive Types	Objects
Directly store values	Store data and behavior
No <code>new</code> required	Usually created with <code>new</code>
Lightweight	Additional object overhead
More efficient	More powerful and flexible

Class vs. Object

- A **class** defines a new **data type** — a logical framework describing member relationships
- An **object** is an **instance** of a class — it has **physical reality** (occupies memory)

	Class	Object
Type	Blueprint / Template	Instance of a class
Nature	Logical construct	Physical reality
Memory	None allocated	Memory allocated
Example	Box	mybox

Relationship Between Class and Object

Concept	Description
Class	Blueprint or template
Object	Real instance created from the class

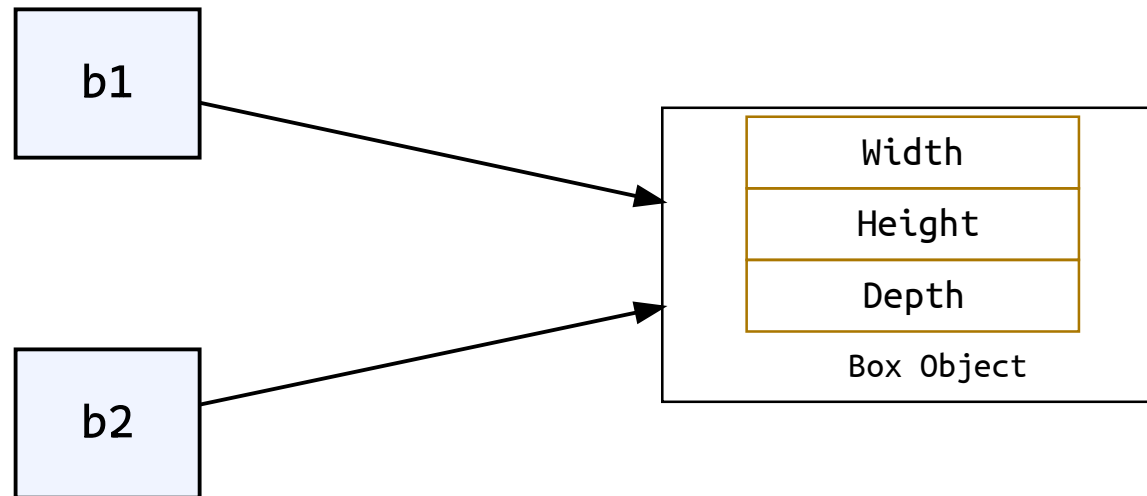
Example

- Class: `Car`
- Objects: `myCar`, `sportsCar`

```
Car myCar = new Car();  
Car sportsCar = new Car();
```

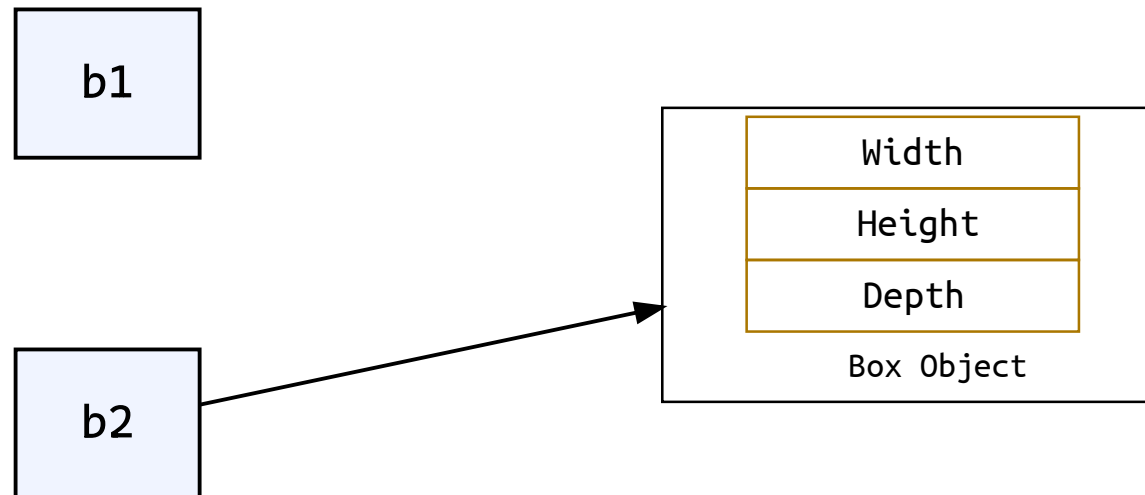
Assigning Object Reference Variables

```
Box b1 = new Box();  
Box b2 = b1; // b2 now references the same object as b1
```



Assigning Object Reference Variables

```
Box b1 = new Box();  
Box b2 = b1; // b2 now references the same object as b1  
// ...  
b1 = null; // b1 no longer references the object
```



Introducing Methods

- A class typically contains:
 - **Instance variables** → data
 - **Methods** → behavior
- Methods define the **actions** an object can perform

General Form of a Method

```
returnType methodName(parameterList) {  
    // method body  
}
```

Components

Component	Purpose
<code>returnType</code>	Type of value returned
<code>methodName</code>	Method identifier
<code>parameterList</code>	Inputs to the method
Method Body	Statements executed by the method

Return Type

- Specifies the type of value returned by the method
- Can be:
 - Primitive types (`int`, `double`, etc.)
 - Class types (`String`, `Box`, etc.)
- Use `void` if no value is returned

Examples

```
int getAge() { ... }  
  
String getName() { ... }  
  
void display() { ... }
```

Method Name

- Identifies the method
- Must be a *valid* Java **identifier**
- Should be meaningful and descriptive
- Conventionally uses **camelCase** naming style

Examples

```
calculateArea()  
displayInfo()  
getVolume()
```

Adding a Method to the Box Class

```
1  class Box {  
2      double width;  
3      double height;  
4      double depth;  
5      // display volume of a box  
6      void volume() {  
7          IO.print("Volume is ");  
8          IO.println(width * height * depth);  
9      }  
10 }
```

Example Method Call

```
1  class BoxDemo3 {
2      public static void main(String[] args) {
3          Box mybox1 = new Box();
4          Box mybox2 = new Box();
5          // assign values to mybox1's instance variables
6          mybox1.width = 10;
7          mybox1.height = 20;
8          mybox1.depth = 15;
9          // assign different values to mybox2's instance variables
10         mybox2.width = 3;
11         mybox2.height = 6;
12         mybox2.depth = 9;
13         // display volume of first box
14         mybox1.volume();
15         // display volume of second box
16         mybox2.volume();
17     }
18 }
```

Example Method Call

```
1  class BoxDemo3 {  
2      public static void main(String[] args) {  
3          Box mybox1 = new Box();  
4          Box mybox2 = new Box();  
5          // assign values to mybox1's instance variables  
6          mybox1.width = 10;  
7          mybox1.height = 20;  
8          mybox1.depth = 15;  
9          // assign different values to mybox2's instance variables  
10         mybox2.width = 3;  
11         mybox2.height = 6;  
12         mybox2.depth = 9;  
13         // display volume of first box  
14         mybox1.volume();  
15         // display volume of second box  
16         mybox2.volume();  
17     }  
18 }
```

Console Output

```
Volume is 3000.0  
Volume is 162.0
```

Accessing Class Members

- **Members** of a class include:
 - Instance variables
 - Methods
- How a member is accessed depends on **where the access occurs**

Access from Outside the Class

- Must use an **object reference**
- Access is performed using the **dot operator** **.**

Example

```
Box mybox = new Box();  
  
mybox.width = 10;  
IO.println(mybox.width);
```

- Syntax:

```
object.member
```

Access from Inside the Same Class

- Members can be accessed **directly**
- **No** object name or **dot** operator is required

Example

```
class Car {  
    double speed;  
    void accelerate() {  
        speed += 10; // Accessing speed directly  
    }  
}
```

- **speed** is accessed directly inside the class

The Same Rule Applies to Methods Outside the Class

```
Box mybox = new Box();  
mybox.volume();
```

Inside the Same Class

```
void display() {  
    volume();  
}
```

- Inside a class, methods can call other methods directly

Returning a Value

```
1 class Box {  
2     double width;  
3     double height;  
4     double depth;  
5     // compute and return volume  
6     double volume() {  
7         return width * height * depth;  
8     }  
9 }
```

Calling The **volume** Method

```
1  class BoxDemo4 {
2      public static void main(String[] args) {
3          Box mybox1 = new Box();
4          Box mybox2 = new Box();
5          double vol;
6          // assign values to mybox1's instance variables
7          mybox1.width = 10;
8          mybox1.height = 20;
9          mybox1.depth = 15;
10         // assign different values to mybox2's instance variables
11         mybox2.width = 3;
12         mybox2.height = 6;
13         mybox2.depth = 9;
14         vol = mybox1.volume(); // get volume of first box
15         IO.println("Volume is " + vol);
16         vol = mybox2.volume(); // get volume of second box
17         IO.println("Volume is " + vol);
18     }
19 }
```

Calling The **volume** Method

```
1  class BoxDemo4 {
2      public static void main(String[] args) {
3          Box mybox1 = new Box();
4          Box mybox2 = new Box();
5          double vol;
6          // assign values to mybox1's instance variables
7          mybox1.width = 10;
8          mybox1.height = 20;
9          mybox1.depth = 15;
10         // assign different values to mybox2's instance variables
11         mybox2.width = 3;
12         mybox2.height = 6;
13         mybox2.depth = 9;
14         IO.println("Volume is " + mybox1.volume());
15         IO.println("Volume is " + mybox2.volume());
16     }
17 }
```

Type Compatibility of Objects

```
1 class A { int i; }
2 class B { int j; }
3
4 class C {
5     public static void main(String[] args) {
6         A a = new B();
7     }
8 }
```

- This code will **not compile** because **A** and **B** are **unrelated classes**
- In Java, you cannot assign an object of one class to a reference variable of another class unless there is an **inheritance relationship** between them

Parameters

- Parameters provide input to a method
- Written as **type–identifier** pairs
- Multiple parameters are separated by commas
- Empty parentheses indicate no parameters

Examples

```
void setSize(int w, int h)
```

```
double area(double radius)
```

```
void display()
```

Parameters and Arguments

When values are passed to a method, two related terms are used:

- **Parameter**
 - Defined in the **method declaration**
 - Acts as a placeholder for incoming values
 - Receives data when the method is invoked
- **Argument**
 - The **actual value** passed to the method
 - Supplied when the method is called

Parameter vs Argument

```
void displayAge(int age) {  
    IO.println(age);  
}
```

- **age** is a parameter

```
displayAge(20);
```

- **20** is an argument

Parameter vs Argument

```
void add(int x, int y) {  
    IO.println(x + y);  
}  
  
add(5, 10);
```

Method Definition

Method Call

x, **y** → Parameters

5, **10** → Arguments

Easy Way to Remember

- Parameter $\Rightarrow$ Placeholder
- Argument $\Rightarrow$ Actual Value

Example

```
void greet(String name) { ... }  
  
greet("Alice");
```

- **name** $\rightarrow$ Parameter
- **"Alice"** $\rightarrow$ Argument

Adding a Method That Takes Parameters

```
1  class Box {  
2      double width;  
3      double height;  
4      double depth;  
5      // compute and return volume  
6      double volume() {  
7          return width * height * depth;  
8      }  
9      // sets dimensions of box  
10     void setDim(double w, double h, double d) {  
11         width = w;  
12         height = h;  
13         depth = d;  
14     }  
15 }
```

Calling the **setDim** Method

```
1 class BoxDemo5 {
2     public static void main(String[] args) {
3         Box mybox1 = new Box();
4         Box mybox2 = new Box();
5         double vol;
6         // initialize each box
7         mybox1.setDim(10, 20, 15);
8         mybox2.setDim(3, 6, 9);
9         // get volume of first box
10        vol = mybox1.volume();
11        IO.println("Volume is " + vol);
12        // get volume of second box
13        vol = mybox2.volume();
14        IO.println("Volume is " + vol);
15    }
16 }
```

Calling the **setDim** Method

```
1 class BoxDemo5 {  
2     public static void main(String[] args) {  
3         Box mybox1 = new Box();  
4         Box mybox2 = new Box();  
5         double vol;  
6         // initialize each box  
7         mybox1.setDim(10, 20, 15);  
8         mybox2.setDim(3, 6, 9);  
9         // get volume of first box  
10        vol = mybox1.volume();  
11        IO.println("Volume is " + vol);  
12        // get volume of second box  
13        vol = mybox2.volume();  
14        IO.println("Volume is " + vol);  
15    }  
16 }
```

Console Output

```
Volume is 3000.0  
Volume is 162.0
```

Constructors

- A **constructor** initializes an object when it is created
- Automatically executed during object creation
- Has the **same name** as the class
- Similar to a method, but:
 - Has **no return type**
 - Not even `void`

Constructors

- A **constructor** initializes an object when it is created
- Automatically executed during object creation
- Has the **same name** as the class
- Similar to a method, but:
 - Has **no return type**
 - Not even `void`

Example

```
class Box {  
  
    Box() {  
        IO.println("Box created");  
    }  
}
```

When Is a Constructor Called?

Object Creation

```
Box mybox = new Box();
```

What Happens?

1. Memory is allocated
2. Constructor is executed
3. Object reference is returned
4. Assignment completes

Why Use Constructors?

Without a constructor:

```
Box b = new Box();  
  
b.width = 10;  
b.height = 20;  
b.depth = 15;
```

With a constructor:

```
Box b = new Box(10, 20, 15);
```

Benefits

- Automatic initialization
- Less repetitive code
- Objects start in a valid state

Constructors and **new**

General Form

```
classVar = new ClassName();
```

The parentheses **()** indicate a **constructor call**.

```
new Box()
```

- Creates a **Box** object
- Invokes the **Box()** constructor

Constructor Example

```
1  class Box {  
2      double width;  
3      double height;  
4      double depth;  
5      // This is the constructor for Box.  
6      Box() {  
7          IO.println("Constructing Box");  
8          width = 10;  
9          height = 10;  
10         depth = 10;  
11     }  
12     // compute and return volume  
13     double volume() {  
14         return width * height * depth;  
15     }  
16 }
```

Constructor Example

```
1  class BoxDemo6 {  
2      public static void main(String[] args) {  
3          // declare, allocate, and initialize Box objects  
4          Box mybox1 = new Box();  
5          Box mybox2 = new Box();  
6          double vol;  
7          // get volume of first box  
8          vol = mybox1.volume();  
9          IO.println("Volume is " + vol);  
10         // get volume of second box  
11         vol = mybox2.volume();  
12         IO.println("Volume is " + vol);  
13     }  
14 }
```

Constructor Example

```
1 class BoxDemo6 {  
2     public static void main(String[] args) {  
3         // declare, allocate, and initialize Box objects  
4         Box mybox1 = new Box();  
5         Box mybox2 = new Box();  
6         double vol;  
7         // get volume of first box  
8         vol = mybox1.volume();  
9         IO.println("Volume is " + vol);  
10        // get volume of second box  
11        vol = mybox2.volume();  
12        IO.println("Volume is " + vol);  
13    }  
14 }
```

Console Output

```
Constructing Box  
Constructing Box  
Volume is 1000.0  
Volume is 1000.0
```

Default Constructor

If No Constructor Is Defined

```
class Box {  
    double width;  
    double height;  
    double depth;  
}
```

- Java automatically provides a **default constructor**

```
Box() { }
```

Default Values

When the default constructor is used, instance variables receive default values:

Type	Default Value
Numeric (<code>int</code> , <code>double</code> , etc.)	<code>0</code>
<code>boolean</code>	<code>false</code>
Reference types	<code>null</code>

Default Values Example

```
class Test {  
    int x;  
    boolean flag;  
    String name;  
}
```

After:

```
Test t = new Test();
```

- `x` → `0`
- `flag` → `false`
- `name` → `null`

Parameterized Constructors

```
1  class Box {  
2      double width;  
3      double height;  
4      double depth;  
5      // This is the constructor for Box.  
6      Box(double w, double h, double d) {  
7          width = w;  
8          height = h;  
9          depth = d;  
10     }  
11     // compute and return volume  
12     double volume() {  
13         return width * height * depth;  
14     }  
15 }
```

Parameterized Constructors

```
1  class BoxDemo7 {  
2      public static void main(String[] args) {  
3          // declare, allocate, and initialize Box objects  
4          Box mybox1 = new Box(10, 20, 15);  
5          Box mybox2 = new Box(3, 6, 9);  
6          double vol;  
7          // get volume of first box  
8          vol = mybox1.volume();  
9          IO.println("Volume is " + vol);  
10         // get volume of second box  
11         vol = mybox2.volume();  
12         IO.println("Volume is " + vol);  
13     }  
14 }
```

Parameterized Constructors

```
1 class BoxDemo7 {  
2     public static void main(String[] args) {  
3         // declare, allocate, and initialize Box objects  
4         Box mybox1 = new Box(10, 20, 15);  
5         Box mybox2 = new Box(3, 6, 9);  
6         double vol;  
7         // get volume of first box  
8         vol = mybox1.volume();  
9         IO.println("Volume is " + vol);  
10        // get volume of second box  
11        vol = mybox2.volume();  
12        IO.println("Volume is " + vol);  
13    }  
14 }
```

Console Output

```
Volume is 3000.0  
Volume is 162.0
```

The **this** Keyword

- **this** refers to the **current object**
- Available inside instance **methods** and **constructors**
- Represents the **object** that **invoked** the method

Example

```
class Box {  
    double width;  
  
    void show() {  
        IO.println(this.width);  
    }  
}
```

Why Use **this**?

- Access the current object's **members** explicitly
- **Distinguish** instance variables from local variables
- Resolve naming **conflicts**

General Form

```
this.instanceVariable  
this.instanceMethod()
```

A Redundant Use of `this`

```
Box(double w, double h, double d) {  
    this.width = w;  
    this.height = h;  
    this.depth = d;  
}
```

- `this` is not strictly necessary here
- The code would work the same without `this`

Instance Variable Hiding

A **local** variable or parameter can have the same name as an **instance** variable.

Example

```
class Box {  
    double width;  
  
    void setWidth(double width) {  
        width = width;  
    }  
}
```

- The parameter **width** hides the instance variable **width**
- Both occurrences refer to the parameter

Resolving the Conflict with **this**

```
class Box {  
    double width;  
  
    void setWidth(double width) {  
        this.width = width;  
    }  
}
```

Expression	Refers To
width	Method parameter
this.width	Instance variable

Constructor Example

Using **this** allows constructor parameters to have the same names as instance variables.

```
class Box {  
    double width;  
    double height;  
    double depth;  
  
    Box(double width, double height, double depth) {  
        this.width = width;  
        this.height = height;  
        this.depth = depth;  
    }  
}
```

Benefits

- Clear and concise
- Common Java programming style
- Avoids inventing alternate parameter names

⚠ No Default Constructor If You Define a Constructor

```
class A {  
    int i;  
}  
  
class B {  
    public static void main(String[] args) {  
        A a = new A();  
    }  
}
```

- Here, **A** has a default constructor, so **new A()** works.

⚠ No Default Constructor If You Define a Constructor

```
class A {  
    int i;  
    A(int x) { i = x; }  
}  
  
class B {  
    public static void main(String[] args) {  
        A a = new A(); // Error: no default constructor  
    }  
}
```

- Here, **A** has a parameterized constructor
- So the **default constructor** is not generated, and
- **new A()** causes a compile-time error.

Garbage Collection

- Java **automatically** manages memory
- Unused objects are removed through **Garbage Collection (GC)**
- No need to explicitly destroy objects

When Does an Object Become Garbage?

An object becomes eligible for garbage collection when:

- No references point to it
- The program can no longer access it

Example

```
Box b = new Box();  
b = null;
```

- After `b = null`, the object has *no reference*
- The object becomes **eligible** for garbage collection

How Java Differs from Some Languages

⇒ Java

```
Box b = new Box();
```

- Memory allocation is explicit (**new**)
- Memory deallocation is **automatic**

⇒ Traditional C++

```
new    → allocate memory  
delete → release memory
```

- Programmer must **manually** free memory

Important Facts About Garbage Collection

- Garbage collection is performed by the **Java Runtime**
- Runs **automatically** in the background
- Timing is not under direct programmer control
- Different JVM implementations may use different GC strategies

 **Key Point**

Eligible for GC $\neq$ Immediately Collected

What Should Programmers Remember?

- Create objects using `new`
- Do **not** explicitly destroy objects
- Let the JVM manage memory reclamation
- Focus on writing correct programs rather than memory cleanup

A Practical Example

```
1 class Stack {
2     int[] stck = new int[10];
3     int tos;
4     Stack() { // Initialize top-of-stack
5         tos = -1;
6     }
7     void push(int item) { // Push an item onto the stack
8         if(tos==9) IO.println("Stack is full.");
9         else stck[++tos] = item;
10    }
11    int pop() { // Pop an item from the stack
12        if(tos < 0) {
13            IO.println("Stack underflow.");
14            return 0;
15        }
16        else return stck[tos--];
17    }
18 }
```

A Practical Example

```
1 class TestStack {
2     public static void main(String[] args) {
3         Stack mystack1 = new Stack();
4         Stack mystack2 = new Stack();
5         // push some numbers onto the stack
6         for(int i=0; i<10; i++) mystack1.push(i);
7         for(int i=10; i<20; i++) mystack2.push(i);
8         // pop those numbers off the stack
9         IO.println("Stack in mystack1:");
10        for(int i=0; i<10; i++)
11            IO.println(mystack1.pop());
12        IO.println("Stack in mystack2:");
13        for(int i=0; i<10; i++)
14            IO.println(mystack2.pop());
15    }
16 }
```